

REPORTE DE AUDITORIA TECNICA

Evaluacion de Calidad - Proyecto mine-inventory

84 / 100 PUNTUACION GLOBAL	ESTADO: ACEPTABLE CON DETALLES POR CORREGIR Este reporte autoevalua el cumplimiento del proyecto frente a los 12 puntos de control de la rubrica tecnica de desarrollo (Frontend y Backend).
--	--

DETALLE DE EVALUACION

1. 1. Estructura de Componentes de Interfaz (Plantillas)	100 / 100
Excelente. La arquitectura de Django Templates usa herencia de plantillas de forma modular.	
Total de plantillas HTML detectadas: 47	
Plantillas que heredan ({% extends %}): 28	
Plantillas con fragmentos reusables ({% include %}): 14	
Bloques definidos ({% block %}): 29	
Uso de static ({% load static %}): 31	
2. 2. Enrutamiento de vistas y módulos	100 / 100
Enrutamiento correcto mediante urls.py centralizado y modular en Django.	
Archivos urls.py mapeados: 10	
Rutas/endpoints totales definidos en el servidor: 61	
- .\almacenamiento\urls.py	
- .\configuracion\urls.py	
- .\core\urls.py	
- .\devoluciones\urls.py	

- - .\inventario\urls.py
- - .\mantenimiento\urls.py
- - .\pagina_principal\urls.py
- - .\prestamo\urls.py
- - .\reportes\urls.py
- - .\usuario\urls.py

3. 3. Consumo de API REST, carga y errores

70 / 100

Consumo de API detectado, pero algunas llamadas carecen de captura de errores o estados de carga.

- - .\static\js\datatables-init.js (Frontend JS): 1 llamadas (SIN manejo de errores sin estado de carga)
- - .\static\js\datatables.js (Frontend JS): 3 llamadas (Con manejo de errores y estado de carga)
- - .\static\js\datatables.min.js (Frontend JS): 1 llamadas (Con manejo de errores y estado de carga)
- - .\static\js\mantenimiento.js (Frontend JS): 1 llamadas (Con manejo de errores y estado de carga)

4. 4. Estilos y metodologías CSS (BEM / Atómico)

70 / 100

Diseño atómico basado en Bootstrap 5 / metodología BEM aplicado de forma general.

- Archivos CSS en la carpeta estática: 4
- Selectores con estructura BEM (___ o --) detectados: 0
- Estructuras de diseño atómico (clases utilitarias Bootstrap): 989
- Estilos en línea (style='...') en HTML: 902 (se sugiere minimizarlos)

5. 5. Binding de datos reactivo y DOM

100 / 100

Data binding unidireccional/bidireccional reactivo manejado mediante JS y listeners de eventos del DOM.

- Archivos JavaScript escaneados: 19
- Escrituras dinámicas en el DOM (binding de salida): 366

- Escuchadores de eventos de entrada (binding de entrada): 47

6. 6. Validación de formularios

100 / 100

Cumplido. Se realizan validaciones robustas tanto en backend (Django Forms) como en frontend.

- Formularios de Django estructurados (forms.py): 6
- Llamadas a validación segura en vistas (.is_valid()): 19
- Atributos de validación HTML5 en plantillas frontend: 53
- - .\almacenamiento\forms.py
- - .\devoluciones\forms.py
- - .\inventario\forms.py
- - .\mantenimiento\forms.py
- - .\prestamo\forms.py
- - .\usuario\forms.py

7. 7. Configuración limpia por Variables de Entorno

50 / 100

Advertencia: Se detectan datos sensibles persistidos de forma directa en settings.py (ej. contraseñas de email o claves de desarrollo). Múdalos a un archivo .env.

- Archivo .env o .env.example presente: No (pero se detectó configuración de dotenv en código)
- Llamadas a variables de entorno en código (os.getenv/decouple): 5
- [ALERTA] Credencial expuesta: SECRET_KEY expuesta en settings.py
- [ALERTA] Credencial expuesta: EMAIL_HOST_PASSWORD expuesta en settings.py

8. 8. Jerarquía y organización del código fuente

100 / 100

Estructura de directorios altamente organizada bajo el estándar de aplicaciones Django MVT.

- - vistas (views.py): 10
- - modelos (models.py): 10

- - formularios (forms.py): 6

- - rutas (urls.py): 10

- - plantillas (.html): 47

- - recursos estaticos (.css/.js): 23

- - tests (tests.py): 11

9. 9. Pruebas unitarias sobre componentes

75 / 100

Pruebas unitarias básicas detectadas (8 tests). Considera ampliar la cobertura.

- Archivos de prueba detectados: 11

- Clases de Test definidos: 1

- Funciones de prueba (test_*): 8

- - .\almacenamiento\tests.py

- - .\configuracion\tests.py

- - .\core\test_settings.py

- - .\devoluciones\tests.py

- - .\inventariol\tests.py

- - .\mantenimiento\tests.py

- - .\pagina_principal\tests.py

- - .\prestamo\tests.py

- - .\proveedores\tests.py

- - .\reportes\tests.py

- - .\usuario\tests.py

10. 10. Optimización de carga (Lazy loading / split)

50 / 100

Ausencia de optimizaciones de rendimiento de carga en el frontend.

- Imágenes o recursos con carga diferida (loading='lazy'): 0

- Uso de scripts asíncronos o diferidos (async/defer): 0

11. 11. Documentación (JSDoc / Comentarios)

100 / 100

Código fuente ampliamente documentado con estándares JSDoc y Docstrings descriptivos.

- Archivos de código escaneados: 96 Python, 19 JavaScript

- Docstrings de documentación en Python: 72

- Comentarios estilo JSDoc en JavaScript: 2762

12. 12. Adaptabilidad de la interfaz (Responsive)

100 / 100

Diseño completamente responsivo y adaptable mediante Bootstrap Grid y consultas de medios CSS.

- Media Queries detectadas en hojas de estilo CSS: 121

- Uso de contenedores y rejillas responsivas (Bootstrap Grid/Flexbox): 516

RECOMENDACIONES CLAVE PARA MEJORA:

- Asegurar que todas las llamadas de consumo de API REST capturen errores de conexión y manejen estados de carga.

- Reducir el uso de estilos en línea (style="...") en las plantillas HTML (trasladar a CSS).

- Migrar las variables SECRET_KEY y EMAIL_HOST_PASSWORD a variables de entorno (.env).

- Crear o ampliar la cobertura de pruebas unitarias (tests.py) para evaluar el comportamiento de los componentes.

- Añadir loading="lazy" a imágenes y async/defer a scripts.